

TEAM 5 — MERIDIAN RESIDENCES

Project Design Document — Common Contract, Database, API & Integration Rules

Purpose: Give all four developers one shared contract so stories can be developed independently and merged into one application without naming, schema, API, environment-variable, or response-format conflicts.

Recommended implementation baseline: React + FastAPI + PostgreSQL. RAG documents/vector search should follow the team's agreed vector-database approach; if pgvector is used, keep it inside PostgreSQL and do not change the relational contracts below.

1. Source Requirements & Design Boundaries

The case study says the shared starter already contains an agreed data model, API contract, architecture, Dockerfile and docker-compose.yml. Teams should extend the shared baseline rather than redesigning it. The shared baseline uses Guest, Property, Reservation, RatePlan, Folio and Order, with PostgreSQL for transactional data and MongoDB for semi-structured guest preference/document data. React consumes the REST API using hooks and Axios. [filecite2turn1file6L337-L341](#)

For Team 5, the vertical entities are LeaseAgreement, MaintenanceTicket and RecurringInvoice. The resident portal must let residents view their lease, upcoming invoices and submit maintenance requests. Sprint 3 adds a lease/policy RAG assistant, a maintenance triage agent, and a monthly UiPath invoice workflow. [filecite2turn1file0L13-L38](#)

Cloud/deployment requirements remain AWS EC2 and Azure App Service Containers, with CI/CD gates Dev → QA → UAT → Staging → Production, plus monitoring and incident triage. [filecite2turn1file4L238-L249](#) [filecite2turn1file5L270-L285](#)

2. One Application — Four Story Ownership

Member	Story / Ownership	Main frontend	Main backend	Shared dependency
Member 1	View Lease Details	Lease.jsx	leases router/service/model	Guest + Property + LeaseAgreement
Member 2	View & Generate Recurring Invoices	Invoices.jsx	invoices router/service/model	LeaseAgreement
Member 3	Submit, Track & Triage Maintenance	Maintenance.jsx	maintenance router/service/model + triage agent	LeaseAgreement / unit
Member 4	Resident Lease & Policy AI Assistant	Assistant.jsx	assistant router + RAG services	Guest / lease context + documents

Rule: each member may own and implement their story independently, but shared names, IDs, enums, URL prefixes, JSON fields, database conventions and environment-variable names in this document are frozen unless the whole team agrees.

3. Shared Repository Structure

```
meridian-residences/
├── README.md
├── .env.example
├── docker-compose.yml          # use starter version; extend only if required
├── frontend/
│   └── src/
│       ├── App.jsx
│       └── main.jsx
```

```

├── components/
├── pages/
│   ├── Login.jsx
│   ├── Dashboard.jsx
│   ├── Lease.jsx
│   ├── Invoices.jsx
│   ├── Maintenance.jsx
│   ├── Assistant.jsx
│   └── Profile.jsx
├── services/
│   ├── api.js
│   ├── leaseService.js
│   ├── invoiceService.js
│   ├── maintenanceService.js
│   └── assistantService.js
├── hooks/
├── context/
├── styles/
├── backend/
│   └── app/
│       ├── main.py
│       ├── core/
│       ├── models/
│       ├── schemas/
│       ├── routers/
│       ├── services/
│       ├── ai/
│       └── tests/
├── database/
│   ├── migrations/
│   └── seed/
├── documents/
│   ├── lease_terms/
│   └── building_policies/
├── automation/uipath/
├── deployment/aws/
├── deployment/azure/
├── monitoring/
└── .github/workflows/ci-cd.yml

```

4. Naming & Coding Rules — FREEZE THESE

Area	Rule	Example
DB tables	snake_case, plural	lease_agreements
DB columns	snake_case	monthly_rate
Python files	snake_case	lease_service.py
Python classes	PascalCase	LeaseAgreement
React files	PascalCase.jsx	Lease.jsx
JS variables	camelCase	monthlyRate
API paths	lowercase plural nouns	/api/v1/leases
JSON fields	snake_case	billing_period_start
Primary key	UUID string	id
Foreign key	<entity>_id	lease_id
Dates	ISO 8601	2026-09-09T10:30:00Z
Money	decimal in DB; JSON number	12500.00
Timestamps	UTC	created_at / updated_at
Booleans	true/false	is_active

Do not create variants such as leaseId vs lease_id, maintenanceTickets vs maintenance_tickets, invoiceStatus vs payment_status, or /leases vs /lease. Use this contract.

5. Shared Database Design

The original baseline defines Guest, Property, Reservation, RatePlan, Folio and Order as shared entities. Their baseline fields are retained; the Team 5 tables below extend the baseline for serviced-apartment operations.

5.1 Core/shared entities

Table	Required fields used by Team 5	Relationship / note
guests	id UUID PK, name, email, phone, loyalty_tier, created_at	Shared baseline; lease points to guest
properties	id UUID PK, name, brand, address, timezone	Shared baseline; building/property
reservations	id UUID PK, guest_id FK, property_id FK, check_in, check_out, status, rate_plan_id	Shared baseline; do not redesign
rate_plans	id UUID PK, property_id FK, name, nightly_rate, cancellation_policy	Shared baseline
folios	id UUID PK, reservation_id FK, line_items, balance, status	Shared baseline; not the recurring-invoice replacement
orders	id UUID PK, property_id FK, guest_id nullable, items, total, placed_at	Shared baseline

5.2 Team 5 entity: lease_agreements

Column	Type	Null?	Rule
id	UUID	NO	PK
unit_id	UUID	NO	FK to units
guest_id	UUID	NO	FK to guests
start_date	DATE	NO	Lease term start
end_date	DATE	NO	Lease term end
monthly_rate	NUMERIC(12,2)	NO	Positive amount
renewal_date	DATE	YES	Next renewal date
status	VARCHAR	NO	active / expired / terminated / pending
created_at	TIMESTAMPTZ	NO	UTC
updated_at	TIMESTAMPTZ	NO	UTC

5.3 Team 5 entity: units

The case brief refers to a unit in LeaseAgreement and MaintenanceTicket. A unit table is therefore required as the concrete apartment/unit resource behind those references.

Column	Type	Null?	Rule
id	UUID	NO	PK
property_id	UUID	NO	FK to properties
unit_number	VARCHAR(50)	NO	Unique within property
unit_type	VARCHAR(50)	YES	Studio / 1BHK / 2BHK etc.; project data
status	VARCHAR	NO	available / occupied / maintenance
created_at	TIMESTAMPTZ	NO	UTC

5.4 Team 5 entity: recurring_invoices

Column	Type	Null?	Rule
id	UUID	NO	PK
lease_id	UUID	NO	FK to lease_agreements

billing_period_start	DATE	NO	Inclusive
billing_period_end	DATE	NO	Inclusive
amount	NUMERIC(12,2)	NO	Invoice amount
due_date	DATE	NO	Payment due date
payment_status	VARCHAR	NO	pending / paid / overdue / cancelled
generated_at	TIMESTAMPTZ	NO	UTC
sent_at	TIMESTAMPTZ	YES	When sent
paid_at	TIMESTAMPTZ	YES	When paid
created_at	TIMESTAMPTZ	NO	UTC
updated_at	TIMESTAMPTZ	NO	UTC

5.5 Team 5 entity: maintenance_tickets

Column	Type	Null?	Rule
id	UUID	NO	PK
unit_id	UUID	NO	FK to units
guest_id	UUID	NO	FK to guests
issue_type	VARCHAR(50)	NO	plumbing / electrical / hvac / appliance / other
description	TEXT	NO	Resident issue
priority	VARCHAR(20)	NO	low / medium / high / urgent
status	VARCHAR(30)	NO	open / assigned / in_progress / resolved / cancelled
vendor_queue	VARCHAR(100)	YES	Assigned queue after triage
escalated	BOOLEAN	NO	Default false
created_at	TIMESTAMPTZ	NO	UTC
updated_at	TIMESTAMPTZ	NO	UTC
resolved_at	TIMESTAMPTZ	YES	UTC

5.6 RAG document tables (recommended if vector search is in PostgreSQL)

Table	Columns	Purpose
documents	id UUID PK, title, document_type, source_path, version, created_at	Lease terms/building policy metadata
document_chunks	id UUID PK, document_id FK, chunk_index, content, embedding, metadata JSONB, created_at	Searchable chunks; embedding uses vector type if pgvector is selected

If the starter project already provides MongoDB document storage, do not delete or replace it without team agreement. The case study explicitly assigns MongoDB to semi-structured guest preference/document data, while PostgreSQL is the transactional store. [filecite?turn1file4?L238-L242](#)

6. Relationships / ERD

Guest (1) —< LeaseAgreement >— (1) Unit —< MaintenanceTicket
 |
 —< RecurringInvoice

Property (1) —< Unit
 Property (1) —< Reservation >— (1) Guest

Document (1) —< DocumentChunk

LeaseAgreement

- belongs to Guest
- belongs to Unit
- has many RecurringInvoices

MaintenanceTicket

- belongs to Unit
- belongs to Guest

Important: recurring invoices belong to leases, not folios. Folio remains the shared hospitality baseline object for a stay. Team 5's business context specifically uses recurring lease-like billing rather than a single folio. [filecite?turn1file0?L16-L23?](#)

7. Database Constraints & Indexes

- All primary keys are UUIDs. Generate IDs in the backend/database consistently; never mix integer IDs and UUIDs.
- Foreign keys: lease_agreements.guest_id → guests.id; lease_agreements.unit_id → units.id; units.property_id → properties.id; recurring_invoices.lease_id → lease_agreements.id; maintenance_tickets.unit_id → units.id; maintenance_tickets.guest_id → guests.id.
- Unique constraint: (property_id, unit_number).
- Prevent duplicate invoices for the same lease and billing period: UNIQUE (lease_id, billing_period_start, billing_period_end). This is essential for the UiPath monthly workflow.
- Indexes: lease_agreements(guest_id), lease_agreements(unit_id), lease_agreements(status), recurring_invoices(lease_id), recurring_invoices(payment_status), recurring_invoices(due_date), maintenance_tickets(guest_id), maintenance_tickets(unit_id), maintenance_tickets(status), maintenance_tickets(priority), document_chunks(document_id).
- Use database transactions for invoice generation and maintenance status changes.

8. Frozen Enums / Allowed Values

Enum	Allowed values
lease_status	pending, active, expired, terminated
unit_status	available, occupied, maintenance
invoice_payment_status	pending, paid, overdue, cancelled
maintenance_issue_type	plumbing, electrical, hvac, appliance, general, other
maintenance_priority	low, medium, high, urgent
maintenance_status	open, assigned, in_progress, resolved, cancelled
document_type	lease_terms, building_policy
user_role	resident, staff, admin

If the UI needs a new value, add it centrally after team agreement. Do not silently invent strings in one story.

9. Common API Contract

The case study provides shared REST endpoints for reservations, guests, folios and availability. Keep those endpoints intact. [filecite?turn1file4?L230-L237?](#) Team 5 adds the following vertical endpoints.

9.1 Shared baseline endpoints

Method	Endpoint	Purpose
GET	/api/v1/reservations	List/filter reservations
GET	/api/v1/reservations/{id}	Get reservation with guest + folio detail
POST	/api/v1/reservations	Create reservation
GET	/api/v1/guests/{id}	Get guest profile + preference document
GET	/api/v1/folios/{id}	Get folio

GET	/api/v1/availability	Check availability
-----	----------------------	--------------------

9.2 Team 5 lease APIs

Method	Endpoint	Request	Response
GET	/api/v1/leases/{lease_id}	—	Lease detail + unit + guest summary
GET	/api/v1/guests/{guest_id}/leases	—	Resident's leases
GET	/api/v1/leases/{lease_id}/summary	—	Dashboard lease summary

9.3 Team 5 invoice APIs

Method	Endpoint	Request	Response
GET	/api/v1/invoices	query: lease_id, payment_status	Paginated invoice list
GET	/api/v1/invoices/{invoice_id}	—	Invoice detail
POST	/api/v1/invoices/generate	lease_id + billing period	Creates one invoice idempotently
POST	/api/v1/invoices/generate-batch	billing period + active leases	Generates invoices for active leases
PATCH	/api/v1/invoices/{invoice_id}/payment-status	payment_status	Updated invoice

9.4 Team 5 maintenance APIs

Method	Endpoint	Request	Response
POST	/api/v1/maintenance-tickets	issue_type, description	Created ticket
GET	/api/v1/maintenance-tickets	query: guest_id/status/priority	Paginated tickets
GET	/api/v1/maintenance-tickets/{ticket_id}	—	Ticket detail
PATCH	/api/v1/maintenance-tickets/{ticket_id}	status / priority / vendor_queue / escalated	Updated ticket
POST	/api/v1/maintenance-tickets/{ticket_id}/triage	—	AI triage result

9.5 Team 5 assistant APIs

Method	Endpoint	Request	Response
POST	/api/v1/assistant/chat	message + optional conversation_id	Answer + sources
POST	/api/v1/assistant/reindex	document metadata	Admin/internal reindex result

10. Standard Request / Response Shape

All developers should use one predictable API shape. The frontend service layer should not have to understand five different response formats.

```
Success:
{
  "success": true,
  "data": { ... },
  "message": "Optional human-readable message",
  "meta": { "request_id": "..." }
```

```

}

List:
{
  "success": true,
  "data": [ ... ],
  "meta": {
    "page": 1,
    "page_size": 20,
    "total": 42,
    "request_id": "..."
  }
}

Error:
{
  "success": false,
  "data": null,
  "message": "Lease not found",
  "error_code": "LEASE_NOT_FOUND",
  "meta": { "request_id": "..." }
}

```

11. Standard HTTP Status Codes

Code	Use
200	Successful GET/PATCH/action
201	Successful POST that creates a resource
400	Invalid request
401	Missing/invalid authentication
403	Authenticated but not permitted
404	Resource not found
409	Conflict / duplicate business operation
422	Validation error
500	Unexpected server error

12. Common API Query & Pagination Rules

```

GET /api/v1/invoices?page=1&page_size=20&payment_status=overdue
GET /api/v1/maintenance-tickets?page=1&page_size=20&status=open
GET /api/v1/leases/{id}

```

```

Defaults:
page = 1
page_size = 20
maximum page_size = 100

```

Do not create story-specific pagination parameter names such as limit/offset in one API and page/page_size in another unless the team changes the global contract.

13. Common Frontend Contract

```

// frontend/src/services/api.js
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;

// Axios instance:
// baseURL = API_BASE_URL
// headers = Content-Type: application/json

```

```

// attach auth token in one interceptor
// normalize errors in one place

// Services:
leaseService.getLease(leaseId)
leaseService.getGuestLeases(guestId)

invoiceService.getInvoices(params)
invoiceService.getInvoice(invoiceId)
invoiceService.generateInvoice(payload)

maintenanceService.createTicket(payload)
maintenanceService.getTickets(params)
maintenanceService.getTicket(ticketId)
maintenanceService.triageTicket(ticketId)

assistantService.chat(payload)

```

Rule: pages/components do not call Axios directly. All API calls go through services. This prevents each member from creating a different base URL, error handling pattern or header configuration.

14. Common React State / Variable Names

Purpose	Use this exact naming pattern
Authenticated user	currentUser
User ID	currentUser.id
Guest ID	currentUser.guest_id
Selected lease	selectedLease
Lease list	leases
Invoice list	invoices
Selected invoice	selectedInvoice
Maintenance list	maintenanceTickets
Selected ticket	selectedTicket
Assistant conversation	messages
Assistant input	messageInput
Loading	isLoading / isSubmitting
Error	error
Pagination	page / pageSize / total

Avoid generic names such as data, result, item and response for long-lived state. Use domain names so merged code remains readable.

15. Common Environment Variables

```

# Backend
APP_NAME=meridian-residences
APP_ENV=development
APP_HOST=0.0.0.0
APP_PORT=8000
API_PREFIX=/api/v1
DATABASE_URL=postgresql://<user>:<password>@<host>:5432/<database>
MONGODB_URL=<starter-mongodb-connection-if-used>
MONGODB_DATABASE=meridian_residences
CORS_ORIGINS=http://localhost:5173

# Authentication / security
JWT_SECRET_KEY=<do-not-commit>
JWT_ALGORITHM=HS256

```

```

JWT_EXPIRE_MINUTES=60

# AI / RAG
LLM_PROVIDER=<approved-provider>
LLM_MODEL=<approved-model>
LLM_API_KEY=<do-not-commit>
EMBEDDING_MODEL=<approved-embedding-model>
VECTOR_DB_URL=<if external vector DB is used>

# Frontend
VITE_API_BASE_URL=http://localhost:8000/api/v1

# Automation / integrations
UIPATH_API_URL=<if required>
JIRA_BASE_URL=<if required>
JIRA_PROJECT_KEY=<if required>

# Observability
LOG_LEVEL=INFO

```

Never commit .env. Commit only .env.example with placeholder values. The case study explicitly requires compliance guardrails and says no guest PII should be placed in AI coding-assistant prompts. [filecite:turn1file5L259-L269](#)

16. Authentication & Resident Data Rules

- Every resident API must derive the resident/guest identity from authenticated context where authentication is implemented; do not trust a guest_id supplied by the browser as the only authorization check.
- A resident can read only their own lease(s), invoices and maintenance tickets.
- Staff/admin endpoints may have broader access, but role checks must be centralized.
- Never send guest PII to an LLM unless the approved design explicitly requires it and compliance rules permit it.
- Do not log passwords, tokens, API keys or full sensitive guest data.

17. Story-to-API Ownership Matrix

Story	APIs owned	Tables owned	Frontend
S1 Lease	GET /leases/{id}; GET /guests/{id}/leases; GET /leases/{id}/summary	lease_agreements, units (read), guests (read)	Lease.jsx, LeaseCard
S2 Invoices	GET /invoices; GET /invoices/{id}; POST /invoices/generate; batch; payment status	recurring_invoices	Invoices.jsx, InvoiceCard
S3 Maintenance	POST/GET/PATCH /maintenance-tickets; /triage	maintenance_tickets	Maintenance.jsx, MaintenanceCard
S4 Assistant	POST /assistant/chat; reindex	documents, document_chunks	Assistant.jsx, ChatMessage

Cross-story reads are allowed through APIs, but each member should not directly modify another member's tables or service logic. For example, the invoice service reads LeaseAgreement data through the backend data layer; the invoice developer should not edit the lease router.

18. Business Rules That Must Be Shared

- Only active leases are eligible for monthly recurring-invoice generation.

- Invoice amount is derived from the lease monthly_rate for the defined billing period unless a future agreed rule says otherwise.
- Invoice generation must be idempotent: running the same billing period twice must not create duplicate invoices.
- An invoice from a prior cycle becomes overdue when its due date has passed and it has not been paid.
- Maintenance tickets start with status=open.
- Urgent maintenance issues can be escalated automatically by the triage agent; examples in the brief include no heat and water leak. [filecite?turn1file0?L33-L38](#)
- RAG answers must be grounded in lease-terms/building-policy documents. If the documents do not contain the answer, the assistant should say it cannot find the answer rather than inventing a policy.

19. RAG Contract

```
POST /api/v1/assistant/chat
Request:
{
  "message": "Is parking included?",
  "conversation_id": "optional-uuid"
}

Response:
{
  "success": true,
  "data": {
    "answer": "...",
    "sources": [
      {
        "document_id": "uuid",
        "title": "Building Parking Policy",
        "chunk_id": "uuid"
      }
    ],
    "conversation_id": "uuid"
  },
  "message": null,
  "meta": {"request_id": "uuid"}
}
```

The case specifically requires a resident-facing assistant grounded in lease-terms and building-policy documents, with example questions about early termination and parking. [filecite?turn1file0?L30-L32](#)

20. Maintenance AI Triage Contract

```
Input:
{
  "ticket_id": "uuid"
}

Output:
{
  "ticket_id": "uuid",
  "issue_type": "plumbing",
  "priority": "urgent",
  "vendor_queue": "plumbing-emergency",
  "escalated": true,
  "reason": "Water leak requires urgent escalation."
}
```

The agent's stopping condition is simple: classify the ticket, choose the vendor queue, set escalation when required, update the ticket, and stop. It must not keep looping after a successful update. The brief requires triage by priority/type, vendor assignment, and escalation of urgent issues.

21. Recurring Invoice Automation Contract

UiPath runs on the 1st of each month. It should call the batch-generation API for active leases, then identify/flag overdue invoices from the previous cycle. The case study specifies this exact monthly workflow.

- 1. Start on monthly schedule.
- 2. Determine current billing period.
- 3. POST /api/v1/invoices/generate-batch
- 4. Record generated/duplicate/failed counts.
- 5. Check prior-cycle invoices for overdue status.
- 6. Flag overdue invoices.
- 7. Write an automation log.
- 8. Stop.

The batch endpoint must be safe to retry. This is why the unique billing-period constraint is mandatory.

22. Seed Data Contract

Use the same seed IDs across all developers so local screenshots/tests line up.

Entity	Suggested seed records
Guest	GUEST-001 / demo resident
Property	PROP-001 / Meridian Residences
Unit	UNIT-101 / 101
Lease	LEASE-001 / active
Invoices	INV-001 paid; INV-002 pending; INV-003 overdue
Maintenance	TICKET-001 open/high; TICKET-002 resolved
Documents	lease terms + building parking policy + maintenance policy

Actual database IDs should still be valid UUIDs; the labels above are logical identifiers for documentation/test fixtures, not a requirement to use non-UUID primary keys.

23. Testing Contract

Area	Minimum tests
Lease	get active lease; unauthorized lease blocked; missing lease returns 404
Invoice	list invoices; generate invoice; duplicate generation returns existing/conflict safely; overdue detection
Maintenance	create ticket; default open status; update status; urgent triage escalates; vendor queue assigned
Assistant	answer from document; source returned; no-answer response when context is missing
API	validation errors; 404; 401/403; response shape
Integration	frontend service correctly handles success/loading/error

Every member writes unit tests for their story. Shared integration tests are added after the four stories are merged.

24. Git / Branching Rules

```
main                # stable integrated branch
develop            # team integration branch (if used)
feature/member1-lease
feature/member2-invoices
feature/member3-maintenance
feature/member4-assistant
```

Commit examples:

```
feat(lease): add lease details endpoint
feat(invoice): add recurring invoice generation
feat(maintenance): add ticket triage
feat(assistant): add lease policy RAG
test(invoice): add duplicate generation test
```

- Do not modify shared files unnecessarily.
- Before changing shared API contracts, notify all four members.
- Keep migrations additive and ordered.
- One feature branch should not rename another member's variables/files.
- Merge backend contract changes before frontend integration where possible.

25. Shared Files — Who Can Modify Them

File/Area	Rule
docker-compose.yml	Team/shared setup only; do not independently rewrite
.env.example	Shared; add variables only with agreement
backend/app/main.py	Shared; register routers carefully
backend/app/core/config.py	Shared; common env variables only
backend/app/core/database.py	Shared; one DB connection pattern
frontend/src/services/api.js	Shared; one Axios client
frontend/src/App.jsx	Shared routing only
database/migrations	All members may add story migrations; unique migration names
README.md	Shared; document final integrated behavior

26. Integration Sequence

- Step 1 — Freeze this contract and starter baseline.
- Step 2 — Create/confirm PostgreSQL schema and migrations.
- Step 3 — Confirm common Axios client, API prefix and response envelope.
- Step 4 — Member 1 builds Lease story.
- Step 5 — Member 2 builds Invoice story using the frozen LeaseAgreement contract.
- Step 6 — Member 3 builds Maintenance story using the frozen Unit/Guest contract.
- Step 7 — Member 4 builds RAG assistant and document ingestion.
- Step 8 — Merge and run integration tests.
- Step 9 — Add UiPath monthly invoice automation.
- Step 10 — Deploy to AWS EC2 and Azure App Service, then add CI/CD, monitoring and incident triage.

This sequence matches the case-study progression: Sprint 2 core React workflow and manual cloud deployment; Sprint 3 RAG + agent + automation; final sprint CI/CD, monitoring, incident triage and live demo. [filecite2turn1file52L259-L2852](#)

27. Final Definition of Done

- All four stories work inside one React application.
- All story APIs use /api/v1 and the common request/response envelope.
- All database tables use the frozen names and UUID/UTC conventions.
- Resident authorization prevents cross-resident access.
- Recurring invoice generation is idempotent.
- Maintenance triage has a clear stopping condition.
- RAG responses return document sources and do not invent unsupported policies.
- UiPath monthly invoice generation and overdue flagging work.
- Application runs through the provided Docker setup.
- Application is deployed to AWS EC2 and Azure App Service as required.
- CI/CD gates Dev → QA → UAT → Staging → Production are configured.
- Logging, monitoring and at least one alert are configured.
- Team can demonstrate the simulated monthly invoice-generation incident and document the fix in Jira Service Management. [filecite?turn1file9?L483-L485?](#)

28. Quick Developer Checklist — Before You Start Coding

Check	Done?
I use the exact table/entity names from this document	☐
I use UUID IDs and UTC timestamps	☐
I use /api/v1/... paths	☐
I use snake_case JSON fields	☐
I use the common response envelope	☐
I use the shared Axios client instead of direct Axios calls in pages	☐
I do not modify another member's story files	☐
I add my migration and unit tests	☐
I do not hard-code secrets	☐
I do not put guest PII into coding-assistant prompts	☐
I test my API independently before merge	☐
I test the complete application after merge	☐

29. Important Note on the Baseline

This document is the Team 5 implementation contract, not a replacement for the starter repository. The case study says the starter already has the shared data model/API/architecture and Docker setup. Therefore, when the actual starter repo is inspected, preserve its existing names and contracts wherever they match this document. If a starter implementation differs from this design, the team should reconcile that difference once, centrally, before four developers begin independent work.

[filecite?turn1file6?L337-L341?](#)